DEALFLOW360

Complete Project Workflow & Implementation Guide

A beginner-friendly, end-to-end blueprint for the quotation-to-cash sales operations platform

ASP.NET Core • C# • Entity Framework Core • SQL Server • React • JWT

Quotation → Governance → Approval → Negotiation → Fulfillment → Billing → Health → Reporting

Project purpose

DealFlow360 is designed as a self-governing B2B sales operations platform: it does more than create a quote. It enforces pricing discipline, calculates margin and risk, routes approvals, reacts to inventory, supports hybrid billing, allows customer negotiation through a restricted portal, and exposes deal-health signals and reports.

Core principle: anything affecting money, approval, inventory, customer commitment, or billing belongs to backend business logic and must be independently testable.

Prepared as a consolidated project guide from the provided DealFlow360 problem statement and backend blueprint.

1. Document Map

No.	Section
01	Project vision & success criteria
02	System architecture & request flow
03	Users, roles & security boundary
04	Complete end-to-end lifecycle
05	Quotation & margin calculation
06	Discount governance & blended risk
07	Approval routing & audit
08	Upsell / cross-sell recommendations
09	Fulfillment, warehouses & backorders
10	Hybrid billing & subscription proration
11	Customer portal & negotiation
12	Deal health, anomalies & notifications
13	API/controller registry
14	Frontend screens & UX workflow
15	Data model & backend structure
16	Testing & eight-step verification
17	Five-minute demo script
18	Definition of done & implementation checklist
19	Beginner learning path & coding mental model

How to use this guide

- Read Sections 1–4 first to understand the whole system before writing individual screens.
- Use Sections 5–12 when implementing or debugging a business rule.
- Use Section 13 as the API contract checklist and Section 14 as the frontend screen map.
- Use Sections 16–18 immediately before the hackathon demo.
- Keep the request flow in mind: React → HTTP → Controller → Service → Engine → DbContext → SQL Server → DTO → React.

2. Project Vision & Success Criteria

The business problem

Real B2B sales are rarely a simple quote → order → invoice flow. The provided challenge focuses on discount approvals, partial stock across warehouses, mixed one-time and recurring products, customer negotiation, and deal slippage visibility.

The target solution

DealFlow360 turns these situations into controlled workflows. The platform becomes the decision-making layer between the sales user and the operational systems.



Required outcomes

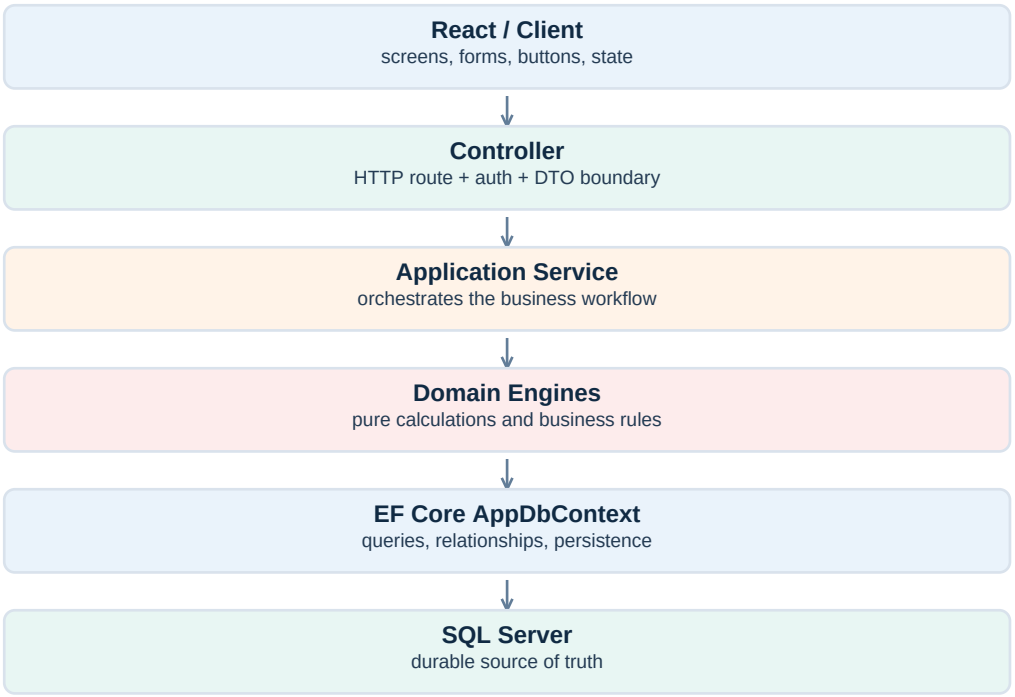
- Sales rep logs in, builds a quotation, and receives the correct approval route based on discount and customer tier.
- Upsell/cross-sell recommendations appear with margin impact while the quote is being built.
- Orders can split across warehouses based on stock, with manual override and backorder handling.
- One order can contain one-time products and recurring subscription lines with separate billing treatment and proration.
- Managers can see stalled deals, discount anomalies, and overall deal health.
- Customers can negotiate the live quotation through a separate, restricted portal.

The challenge explicitly prioritizes business logic, data modeling, and end-to-end workflow over a particular technology vendor.

3. System Architecture & Request Flow

The mental model

Think of the application as a chain of responsibility. Each layer has one job and passes a controlled result to the next layer.



Example: user clicks “Submit for Approval”

- React sends POST /api/quotations/{id}/submit-approval.
- The controller validates the request shape and authorization.
- QuotationService loads the quotation, customer, lines, products and rules.
- Margin, discount-governance and risk engines calculate authoritative values.
- ApprovalRoutingEngine decides the required approval level.
- The service changes quotation state and writes the audit record.
- EF Core persists the result to SQL Server.
- The API returns a DTO; React updates the UI.

Important boundary

React should display and collect input, not become the authority for price, cost, margin, approval, inventory or billing decisions.

Suggested backend structure

Models/ → entity models. Data/ → AppDbContext and initialization. Services/Engines/ → deterministic business engines. Services/ → workflow orchestration. Controllers/ → REST API. Middleware/ → global exception and concurrency handling.

4. Users, Roles & Security Boundary

Role	Primary responsibility	Important permissions
Admin	Master data & configuration	Users, products, warehouses, discount/approval rules, reporting
SalesRep	Build and submit quotations	Create/edit quotes, lines, margin view, submit approval
SalesManager	Approve and manage pipeline	Approval queue, approve/reject/revise, deal health
FinanceOperations	Level-2 approval + operations	Finance approval, fulfillment allocation, payments, credit notes
Customer	Portal collaboration	Own quotes/orders/invoices, comments, counter-offers, confirmation

Zero-leak customer boundary

The customer portal must use a customer-safe DTO. Internal operational intelligence must never be exposed merely because the underlying quotation contains it.

Field	Customer portal
CostTotal / CostPrice	EXCLUDED
MarginAmount / MarginPercent	EXCLUDED
RiskScore	EXCLUDED
ApprovalStatus	EXCLUDED
ManagerRemarks	EXCLUDED

Authentication model

- Internal users authenticate with standard credentials and receive JWT bearer authentication.
- Authorization is role-based at controller/endpoint boundaries.
- Customer portal access is separated from internal operational views and can use a portal/magic-link token boundary.
- Unauthorized users must be blocked by backend authorization rather than by hiding frontend buttons.

5. Complete End-to-End Lifecycle

The entire product can be understood as one continuous stateful business story.

Step	Stage	What the system does
1	Login	User identity and role are established.
2	Configure	Products, price lists, tiers, rules, warehouses and plans exist.
3	Create Quote	Sales rep opens a draft quotation.
4	Add Lines	Backend resolves real product/price/cost/tax information.
5	Calculate	Margin, discount, tax and totals are recomputed.
6	Govern	Line ceilings and blended discount risk are evaluated.
7	Approve	Auto-approval, manager approval or finance escalation is selected.
8	Negotiate	Customer can counter terms through the restricted portal.
9	Re-approve	Changed commercial terms trigger governance again when required.
10	Confirm	Approved final terms become the order commitment.
11	Fulfill	Stock is allocated across warehouses; missing stock becomes backorder.
12	Bill	One-time invoice and recurring billing schedule are generated.
13	Pay	Payments update invoice/outstanding state.
14	Monitor	Deal health and anomaly signals remain visible.
15	Report	Managers/admins review pipeline and sales performance.

Core mental formula

DATA → BUSINESS RULE → CALCULATION → DECISION → STATE CHANGE → DATABASE → NEXT BUSINESS PROCESS

6. Quotation & Margin Calculation

Why the backend calculates it

The frontend can show live totals, but the backend must recalculate from authoritative product/customer data. This prevents arbitrary unit prices or manipulated margin values from becoming the system of record.

Line-level formulas

Value	Formula
Unit Revenue	UnitPrice × (1 – DiscountPercent / 100)
Line Net Amount	Unit Revenue × Quantity
Tax Amount	Line Net Amount × TaxRate / 100
Line Cost	CostPrice × Quantity
Line Margin	Line Net Amount – Line Cost

Quotation totals

Total	Meaning
SubTotal	Sum of gross line values before discount
DiscountTotal	Sum of gross line discount amounts
TaxTotal	Sum of line tax amounts
GrandTotal	SubTotal – DiscountTotal + TaxTotal
CostTotal	Sum of line cost
MarginAmount	Sum of line margins
MarginPercent	MarginAmount ÷ net revenue × 100

Example

Laptop: UnitPrice ■80,000; CostPrice ■60,000; Quantity 2; Discount 10%; Tax 18%. Unit revenue = ■72,000. Net = ■1,44,000. Tax = ■25,920. Cost = ■1,20,000. Margin = ■24,000. Margin percentage = 16.67%.

Implementation note

Use decimal arithmetic for financial values and round monetary outputs consistently. The provided MarginCalculationEngine calculates line values first, then aggregates quotation totals from the recalculated lines.

7. Discount Governance & Blended Risk

Discount governance is not one global percentage

A customer tier can have a ceiling, while a product category can have a stricter ceiling. For each line, the allowed limit is the smaller of those two values.

Core rule

```
maxAllowedDiscount = min(customerTierMaxDiscount, categoryMaxDiscount)
```

Then: excessDiscount = max(0, requestedDiscount – maxAllowedDiscount).

Blended risk

The risk score combines the worst single-line excess, the weighted order-level excess, and a margin deficit. This prevents a quote from hiding many small violations across different lines.

Component	Weight	Purpose
Peak line violation Δpeak	0.40	Captures the worst individual line breach
Weighted margin loss Δweighted	0.35	Captures total excess discount across the quote
Margin deficit	0.25	Captures risk when gross margin falls below 30%

Risk formula

Risk Score = (0.40 × Δpeak) + (0.35 × Δweighted) + (0.25 × MarginDeficit)

Risk score	Route	Result
< 30	AutoApproved	No manual approval required
30–69	Manager	Sales Manager approval
≥ 70	Finance	Level-2 Finance Operations approval

Concrete blended example

Gold customer: normal tier ceiling 15%. Laptop hardware has a 15% category ceiling; Setup Service has a 10% category ceiling. A 12% laptop discount is within limit, but an 18% service discount is 8 percentage points over its category ceiling. The quote therefore cannot be treated as safe simply because the customer is Gold.

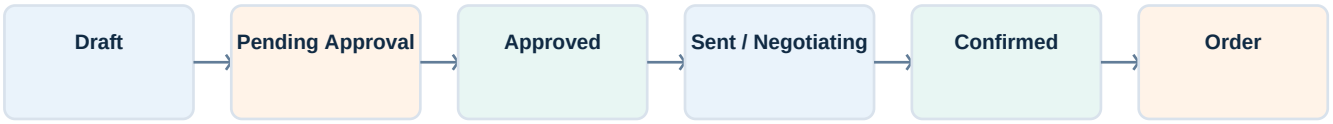
8. Approval Routing, State Changes & Audit

Approval is a state machine, not a button that directly writes “Approved”.

Approval invariants

- A sales rep cannot approve or action their own quotation.
- Reject and Request Revision require meaningful remarks (the blueprint specifies at least 10 characters).
- If a manager approves a quote whose risk is still ≥ 70, it escalates to Finance rather than becoming fully approved.
- Changes to commercial terms after approval must invalidate/re-run the relevant governance and approval process.
- Approval, rejection and edits are audit events with actor, timestamp and reason.

Typical state story



Audit trail

- Record who changed the quotation.
- Record what workflow action happened.
- Record when it happened.
- Record the reason/remarks where the workflow requires it.
- Keep the history durable in the database so a manager can reconstruct the commercial decision.

Backend authority

The service layer should be the place where state transitions are validated. Avoid a generic UpdateStatus endpoint that allows the client to jump between business states.

9. Upsell / Cross-Sell Recommendations

Purpose

While the rep builds a quotation, the system can recommend complementary products based on pairing/co-purchase rules and current promotions, while ensuring the suggestion is commercially healthy.

Ranking logic

```
Margin Delta % = NewQuotationMargin% – CurrentQuotationMargin%

Rank Score = (RuleScore × 0.6) + (Margin Delta % × 0.4)
```

What the UI should show

- Suggested product
- Margin delta if added
- Promotion tag when applicable
- Add to Quote
- Dismiss

Important workflow behavior

After a recommendation is accepted, the quote totals and margin indicator update immediately. The backend remains authoritative when the quote is recalculated or submitted.

10. Fulfillment, Warehouses & Backorders

The problem

A confirmed order may require stock from more than one warehouse. The platform must not pretend that stock exists in one location when the real inventory is distributed.

Greedy allocation algorithm

- Consider active warehouses ordered by ShippingCostWeight ascending.
- For each order line, calculate available stock as OnHand – Reserved.
- Allocate as much as possible from the best eligible warehouse.
- Continue to the next warehouse if quantity remains.
- If total available stock is insufficient, create a Backorder for the remaining quantity.

Fulfillment status logic

Allocation result	Order status
Fully allocated	Allocated
Partially allocated + backorders	PartiallyAllocated
Unallocated balance remains	Confirmed with pending backorder

Replenishment

When new inventory arrives, BackorderConsolidationEngine can find pending backorders for that product and convert available stock into new warehouse allocation records.

Frontend experience

- Show warehouse name and quantity.
- Show estimated shipment count/cost.
- Allow Accept Suggested Split.
- Allow Manual Override.
- Prompt for backorder consolidation when stock arrives.

11. Hybrid Billing & Subscription Proration

One order, two billing behaviors

DealFlow360 separates one-time hardware/services from recurring subscription lines while keeping them on the same commercial order.

Line type	Billing behavior
Hardware / one-time services	Generate immediate commercial invoice
Subscription	Generate recurring BillingSchedule records

Calendar-exact proration

Prorated Charge = (Monthly Unit Rate ÷ Days in Month) × Added Seats × Remaining Active Days

Example

If a customer adds seats during a month, the system uses the actual number of days in that calendar month and the remaining active days to calculate the mid-cycle charge. This is different from simply charging a full month.

Subscription operations

- Show one-time and recurring lines separately.
- Display upcoming recurring billing schedules.
- Support mid-cycle quantity changes with proration.
- Support cancellation/modification workflows and trigger a partial refund or credit note when the configured rules require it.

12. Customer Portal & Negotiation

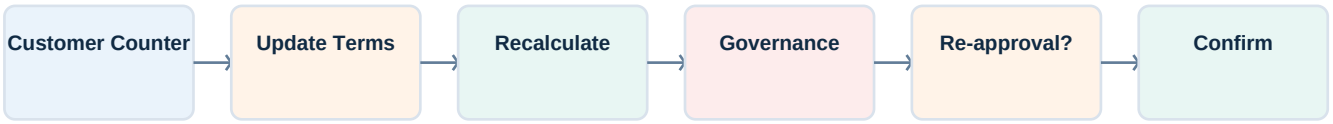
The portal is a separate security boundary

The customer should see the commercial document they need to act on, but not internal cost, margin, risk or manager information.

Customer actions

- View quotation details and current status.
- Comment on a line.
- Request changes.
- Counter a discount.
- Confirm final terms.

Counter-offer workflow



Critical rule

A customer counter-offer is not a trusted final value. The backend updates the affected line terms, recalculates the quote, reruns DiscountGovernanceEngine and BlendedDiscountRiskEngine, and determines whether approval is required again.

Portal endpoint concept

The blueprint exposes a token-based portal route such as /api/portal/quote/{token}, with line comments and counter-offer operations under that restricted context.

13. Deal Health, Anomalies & Notifications

Deal Health Engine

- Stalled deals: quotations in Sent or UnderNegotiation with no activity beyond the configured threshold; the blueprint uses more than five business days.
- Discount anomalies: unusually high discounts on high-value quotes; the blueprint gives >25% discount on >$5,000 subtotal as a concrete rule.
- Overall health: a 0–100 health metric for manager visibility.

Dashboard purpose

The dashboard is not merely a collection of charts. Each alert should help a manager decide what to do next: open the quotation, investigate the discount, follow up on the stalled deal, or address delivery slippage.

Notifications

- Quote status changes
- Approval requests
- Portal inquiries
- Manager alerts and escalations

Operational idea

A notification is a consequence of a business event. The durable event/state belongs in the database first; notification delivery is a separate concern.

14. API & Controller Registry

Controller	Route	Authorization	Main endpoints
AuthController	/api/auth	Anonymous / Authorized	/login, /register, /refresh-token, /me
AdminController	/api/admin	Admin	/users, /customer-tiers, /products, /price-lists, /discount-rules, /warehouses
CustomersController	/api/customers	Rep / Manager / Customer	/, /{id}, /me/quotations, /me/orders, /me/invoices
QuotationsController	/api/quotations	SalesRep / Manager	/, /{id}, /{id}/lines, /{id}/recalculate, /{id}/submit-approval, /{id}/recommendations
ApprovalsController	/api/approvals	Manager / Finance	/pending, /{id}, /{id}/action
FulfillmentController	/api/fulfillment	Operations / Admin	/preview/{orderId}, /allocate/{orderId}, /backorders, /replenish
BillingController	/api/billing	Finance / Admin	/generate-order-billing/{orderId}, /subscriptions/{id}/seat-change
InvoicesController	/api/invoices	Finance / Rep / Customer	/, /{id}, /{id}/pay, /{id}/credit-note
PortalController	/api/portal	Magic-Link Token	/quote/{token}, comments, /counter-offer
DealHealthController	/api/deal-health	Manager / Admin	/summary
ReportsController	/api/reports	Manager / Admin	/dashboard, /pipeline

Endpoint design rule

Endpoints should represent business actions and workflow boundaries. Examples such as /recalculate, /submit-approval, /action, /allocate and /counter-offer are safer than a generic “set any status” endpoint.

15. Frontend Screens & User Experience

Screen	What it proves
Login	Identity, role and protected application entry
Dashboard	Pipeline, health, alerts, revenue and operational attention
Quotations	Draft and active deal discovery
Quotation Builder	Products, quantities, discounts, totals, margin, risk and recommendations
Approval Desk	Risk score, approval steps, actions and audit trail
Pipeline	Deal stage visualization and prioritization
Fulfillment	Warehouse split, stock, manual override and backorder
Billing	Invoice, payment, outstanding amount, recurring schedule and proration
Customer Portal	Restricted quote view, comments, counter-offer and confirmation
Reports	Discount distribution, pipeline and operational alerts
Admin	Products, price lists, discount rules, warehouses, approval chain, subscription plans

Recommended screen-to-API pattern

Keep API calls centralized in modules such as authApi, quotationApi, approvalApi, fulfillmentApi, billingApi, portalApi, reportApi and adminApi. Components should focus on experience and state, while the API layer handles HTTP communication.

UI quality rule

A polished interface is valuable for the hackathon, but it must remain a visualization of real backend state. Do not replace missing backend logic with demo-only JSON when the workflow is supposed to be functional.

16. Data Model & Backend Structure

Domain	Representative records
Identity	User, role, refresh token / auth state
Customers	Customer, CustomerTier
Catalog	Product, category, variants, price lists
Governance	DiscountRule, approval configuration
Sales	Quotation, QuotationLine, approval/audit records
Operations	Warehouse, stock, reservation, allocation, backorder
Billing	Order, Invoice, Payment, CreditNote
Subscriptions	SubscriptionPlan, BillingSchedule, seat changes
Portal	Portal access context, comments, negotiation events
Intelligence	Deal health, anomaly and notification records

Backend folder map

Folder	Responsibility
Models/	Entity models and persistence shape
Data/	AppDbContext, relationships, indexes and initializer
Services/Engines/	Pure deterministic business rules and calculations
Services/	Workflow orchestration and use-case services
Controllers/	REST endpoints and HTTP boundary
Middleware/	Global exception handling and concurrency responses

Source-of-truth principle

SQL Server stores durable business state; EF Core maps the domain model to persistence; engines calculate; services orchestrate; controllers expose.

17. Testing & Eight-Step Verification Flow

#	Test action	Expected proof
1	Log in and configure a discount tier, warehouse and subscription plan	Authentication + admin data work
2	Create quote with a discount above normal allowance	Governance detects excess
3	Submit quote	Approval is requested automatically
4	Accept one upsell suggestion	Total and margin change immediately
5	Approve and allocate stock	Correct warehouse split; two warehouses if required
6	Mix one-time + recurring lines	Invoice and billing schedule are separated correctly
7	Customer requests a larger discount	Quote automatically re-enters approval when needed
8	Confirm, record payment, inspect invoice	Order/billing/payment states update correctly

Unit-test targets

- MarginCalculationEngine: deterministic formulas and rounding.
- DiscountGovernanceEngine: tier/category ceiling combinations.
- BlendedDiscountRiskEngine: threshold routing at <30, 30–69 and ≥70.
- ApprovalRoutingEngine: self-approval prevention, mandatory remarks, escalation.
- WarehouseAllocationEngine: stock split and backorder creation.
- SubscriptionEngine: calendar-day proration.
- CustomerNegotiationEngine: re-calculation and re-approval trigger.
- DealHealthEngine: stalled deals and discount anomalies.

Integration-test target

Run the complete quote-to-payment journey through real HTTP requests and verify the database state after each major transition.

18. Five-Minute Live Demo Script

Time	Scene	What to demonstrate
0:00–0:30	Login	Enter as Sales Rep; show dashboard/workspace.
0:30–1:15	Build quote	Add hardware + service/subscription; show live totals, margin and recommendations.
1:15–1:50	Trigger governance	Apply a high service discount; show risk and automatic manager/finance route.
1:50–2:20	Approve	Manager reviews the risk, approves; show audit trail.
2:20–2:55	Fulfill	Preview multi-warehouse allocation and backorder if stock is short.
2:55–3:25	Bill	Show one-time invoice plus recurring schedule and proration.
3:25–4:10	Customer portal	Open restricted portal, submit counter-offer; show re-approval.
4:10–4:40	Confirm + pay	Confirm terms, record payment, show invoice status.
4:40–5:00	Manager view	Show deal health/anomaly dashboard and report filters.

Demo discipline

- Seed a known scenario so the result is deterministic.
- Avoid manually changing database values during the demo.
- Make every visible decision trace back to a backend rule.
- Prefer two complete end-to-end flows over many disconnected screens.
- Keep the customer portal visibly different and restricted.

19. Definition of Done & Implementation Checklist

Area	Done means
Authentication	JWT/roles work; unauthorized access is blocked.
Admin	Products, tiers, rules, warehouses and plans can be configured.
Quotation	Server-side pricing/totals/margin and audit are authoritative.
Governance	Line ceilings + blended risk + automatic approval route work.
Approvals	Manager and finance sequence works with valid state transitions.
Upsell	Recommendations are data-driven and show margin impact.
Fulfillment	Stock-aware split, manual override and backorder work.
Billing	One-time + recurring billing and proration work.
Portal	Customer view is restricted and negotiation is real.
Negotiation	Changed terms trigger recalculation and re-approval when needed.
Health	Stalled/risky deals are visible to managers.
Reports	Pipeline/discount/performance reporting is usable.
Audit	Important workflow changes are recorded.
Testing	Business engines and end-to-end flow are verified.
Demo	At least two complete flows can be shown reliably.
Docs	Architecture, setup, workflow and next steps are documented.

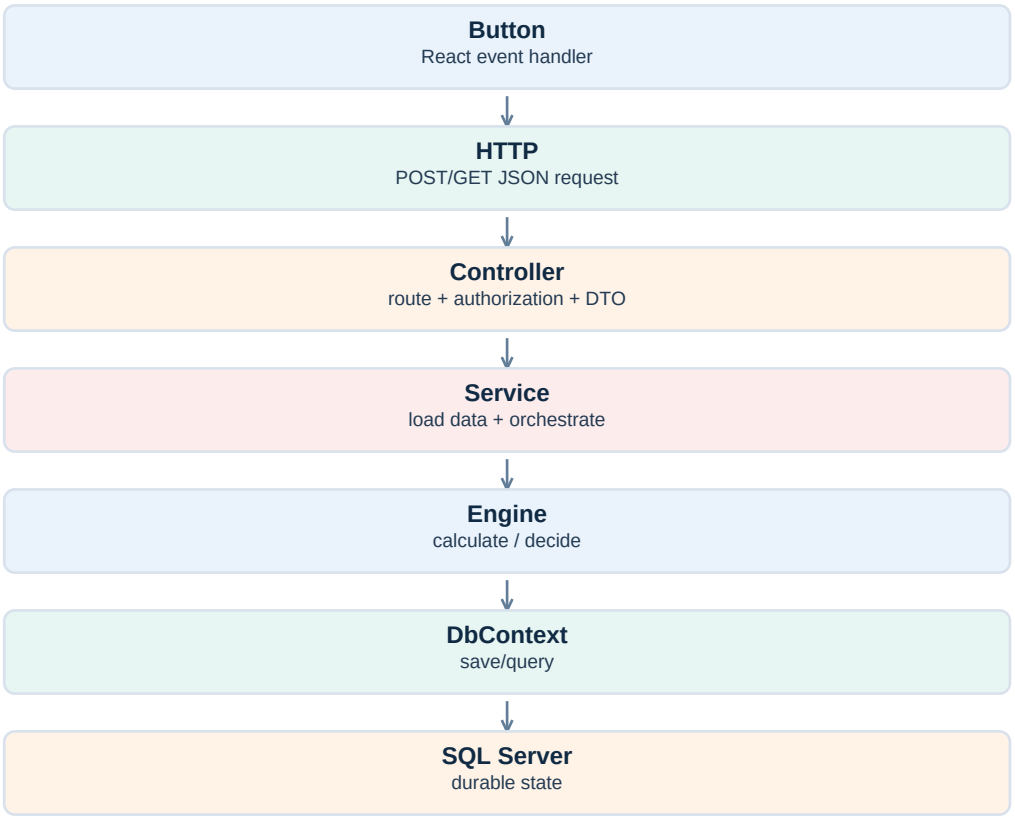
Common mistakes to avoid

- Hardcoding approval outcomes.
- Ignoring category discount ceilings.
- Trusting arbitrary unit prices or totals from the frontend.
- Faking warehouse splits instead of using stock.
- Mixing recurring and one-time billing logic.
- Allowing overpayment or negative payment.
- Using a generic status update to bypass workflow transitions.
- Exposing cost/margin/risk/manager remarks in the customer portal.
- Changing an approved quote without invalidating/rechecking approval.
- Building AI or extra screens before the core business workflow works.

20. Beginner Learning Path & Coding Mental Model

#	Topic	What to understand
1	C# basics	classes, interfaces, methods, properties, decimal, loops, LINQ
2	ASP.NET Core	Program.cs, DI, middleware, controllers, routing
3	DTOs + validation	request/response contracts and safe data boundaries
4	EF Core + SQL Server	DbContext, DbSet, relationships, migrations, queries
5	Authentication	JWT, claims, roles and authorization
6	Quotation	entities, services, margin engine, recalculation
7	Governance	discount rules, risk scoring, approval state machine
8	Operations	warehouse allocation, backorders, billing, proration
9	Portal	restricted DTOs, negotiation and re-approval
10	Frontend integration	API clients, state, forms, loading/errors, protected routes
11	Testing	unit tests for engines + end-to-end business flow

One API request to memorize



Then the response travels back upward: SQL Server → DbContext → Service → Controller → JSON → React state → UI.

21. One-Page DealFlow360 Cheat Sheet

Keep this page open while coding or demoing.

Question	Answer
What is DealFlow360?	A self-governing sales operations platform for quotation-to-cash.
Who owns business authority?	ASP.NET backend service/engine layer.
Where is durable state?	SQL Server via EF Core AppDbContext.
What calculates money?	Backend engines, not the frontend.
What controls discounts?	Customer tier + category ceiling + blended risk.
Risk <30?	AutoApproved.
Risk 30–69?	Manager approval.
Risk ≥70?	Finance approval.
What happens after customer counter-offer?	Recalculate + rerun governance/risk + re-approve if needed.
How does fulfillment work?	Greedy stock-aware multi-warehouse allocation + backorders.
How does hybrid billing work?	One-time invoice + recurring billing schedule.
How is proration calculated?	Monthly rate ÷ days in month × added seats × remaining active days.
What must the customer never see?	Cost, margin, risk, approval status and manager remarks.
What should every important change have?	A valid state transition and audit trail.
What is the best demo?	One complete quote-to-payment flow plus customer negotiation/re-approval.

Final engineering rule

Anything that affects money, approval, inventory, customer commitment, or billing result belongs in backend service-layer business logic and must be independently testable. React is the experience layer; ASP.NET is the business authority; SQL Server is the durable source of truth.

Source note

This guide consolidates the provided DealFlow360 problem statement and the supplied Master Backend Blueprint. It preserves their terminology and workflow rules while reorganizing them into a single implementation and demo document.